

Entwurf digitaler Systeme

2. Vorbereitende Grundlagen

2. Vorbereitende Grundlagen

2.1 Zahlensysteme

Eine Zahl z lässt sich in einem polyadischen Zahlensystem in Potenzen der Basis B zerlegen.

$$z = \sum_{i=N-1}^{-m} b_i \cdot B^i = b_{N-1} \cdot B^{N-1} + b_{N-2} \cdot B^{N-2} + \dots$$

$$\dots + b_0 \cdot B^0 + b_{-1} \cdot B^{-1} + \dots + b_{-m} \cdot B^{-m}$$

b_i = Stellenwert der Stelle i $b_i, \quad 0 \leq b_i \leq B-1$

B = Basis des Zahlensystems

B^i = Stellenfaktor der Stelle i

2.1 Zahlensysteme

Vereinfachte Darstellung

Gewöhnlich verwenden wir nicht die Summengleichung, sondern eine vereinfachte Schreibweise – die so genannte *Positionsschreibweise*. Sie bietet den Vorteil einer kompakten Schreibweise mit gleichzeitiger Einführung des Dezimalpunktes.

$$z = (b_{N-1} b_{N-2} \dots b_0 . b_{-1} b_{-2} \dots b_{-m})$$

In der Regel wird eine Zahl in Positionsschreibweise noch mit Ihrer zugrunde liegenden Basis indiziert, um sicherzugehen, dass es beim Lesen nicht zu Fehlinterpretationen kommt.

$$z = (b_{N-1} b_{N-2} \dots b_0 . b_{-1} b_{-2} \dots b_{-m})_B$$

2.1 Zahlensysteme

Das Dual-, Hexadezimal- und Dezimalsystem sind dominierend.

- Das Dualsystem besitzt die Basis $B = 2$ mit dem Zeichenvorrat $b_i = (0,1)$.
 - Beispiel: $(1010)_2 = 2^3 + 2^1 = (10)_{10}$
- Das Dezimalsystem besitzt die Basis $B = 10$ mit dem Zeichenvorrat $b_i = (0,1,2,3,4,5,6,7,8,9)$.
 - Beispiel: $(123)_{10} = 1 \cdot 10^2 + 2 \cdot 10^1 + 3 \cdot 10^0 = (1111011)_2$
- Das Hexadezimalsystem besitzt die Basis $B = 16$ mit dem Zeichenvorrat $b_i = (0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F)$.
 - Beispiel: $(1F)_{16} = 1 \cdot 16^1 + 15 \cdot 16^0 = (31)_{10}$

2.1.1 Kodierung und Konvertierung

- Wir wollen die Konvertierungsschemata anhand von Beispielen erläutern, da dieses Vorgehen der Praxis entspricht. Natürlich lassen sich alle verwendeten Beispiele auch – mehr oder weniger – geschlossen mathematisch beschreiben, jedoch sieht man dann manchmal „den Wald vor lauter Bäumen nicht“.
- **Beispiel** (Umwandlung von Binärzahlen in Hexadezimalzahlen)
 - Gruppenverfahren: Bilde je nach Basis B' entsprechende Gruppen und fasse die Werte der Basis B in einem *neuen Positionssystem* zusammen.

$$(1010 \quad 1100 \quad 0110)_2 = \underbrace{1010}_A \underbrace{1100}_C \underbrace{0110}_6 = (AC6)_{16}$$

- Manchmal werden für Hexadezimalzahlen auch folgende Schreibweisen gewählt: AC6h oder AC6hex.

2.1.1 Kodierung und Konvertierung

- **Beispiel** (Umwandlung von Hexadezimalzahlen in Binärzahlen)

$$(7563)_{16} = \begin{matrix} 7 & 5 & 6 & 3 \\ \underbrace{} & \underbrace{} & \underbrace{} & \underbrace{} \\ 0111 & 0101 & 0110 & 0011 \end{matrix} = (0111 \ 0101 \ 0110 \ 0011)_2$$

- **Beispiel** (Umwandlung von Binärzahlen in Dezimalzahlen)

- Eine spezielle Umrechnung wird für das Dezimalsystem benötigt. Hier ist es zunächst nur möglich eine *direkte* Berechnung durchzuführen.

$$\begin{aligned} (0010 \ 1010)_2 &= 0 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + \dots \\ &\dots + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = (42)_{10} \end{aligned}$$

$$(1111011)_2 = 2^6 + 2^5 + 2^4 + 2^3 + 2^1 + 1 = (123)_{10}$$

$$\text{Ergebnis: } (1111011)_2 = (123)_{10}$$

2.2.1 Addition

Die Rechenregeln des dezimalen Zahlensystems können grundsätzlich auch auf das duale Zahlensystem angewendet werden.

Beispiel Addition:

	1111011 (123) ₁₀	Augend
+	0000011 (3) ₁₀	Addend
Carry	11 (Übertrag)	
	1111110 (126) ₁₀	Summe

Ein Übertrag (Carry) entsteht immer dann, wenn die jeweilige Spaltensumme $\geq B$ ist.

2.2.1 Addition

Beispiel:

$$\begin{array}{r}
 1111011 \text{ (123)}_{10} \text{ (7Bit)} \\
 + 0001011 \text{ (11)}_{10} \text{ (7Bit)} \\
 \hline
 \text{Carry } 1111 \text{ 11} \\
 10000110 \text{ (134)}_{10} \text{ (8 Bit)}
 \end{array}$$

Das MSB erzeugt ebenfalls einen Übertrag. Somit wird das Ergebnis um ein Bit erweitert.

2.2.2 Subtraktion

Beispiel Subtraktion:

	1111011	(123) ₁₀	Minuend
-	0001111	(15) ₁₀	Subtrahend
Borrow	11	(Borgen)	
	1101100	(108) ₁₀	Differenz

Ein Borger (Borrow) entsteht, wenn der Spaltensubtrahend größer ist als der Spaltenminuend.

2.2.2 Subtraktion

Beispiel:

$$\begin{array}{r}
 1111011 \text{ (123)}_{10} \\
 - 0001011 \text{ (11)}_{10} \\
 \hline
 \text{Borrow} \\
 1110000 \text{ (112)}_{10}
 \end{array}$$

Der Spaltensubtrahend ist hier immer $\leq$ dem Spaltenminuend, so daß kein Borrow-Bit entsteht.

2.2.2 Subtraktion

Unter dem Gesichtspunkt einer Hardware-Implementierung muss ein Rechenwerk beide Operationen (Addition und Subtraktion) ausführen können.

Um die Subtraktion auf eine Addition zurückführen zu können, wird der **Subtrahend** durch sein **Komplement K** (Ergänzung) ersetzt.

$$\bar{b} = K - b, \rightarrow -b = \bar{b} - K, \quad K = 2^N - 1 ; \quad N = \text{Stellenanzahl}$$

$$a - b = a + (-b) \stackrel{!}{=} a + \bar{b} - K$$

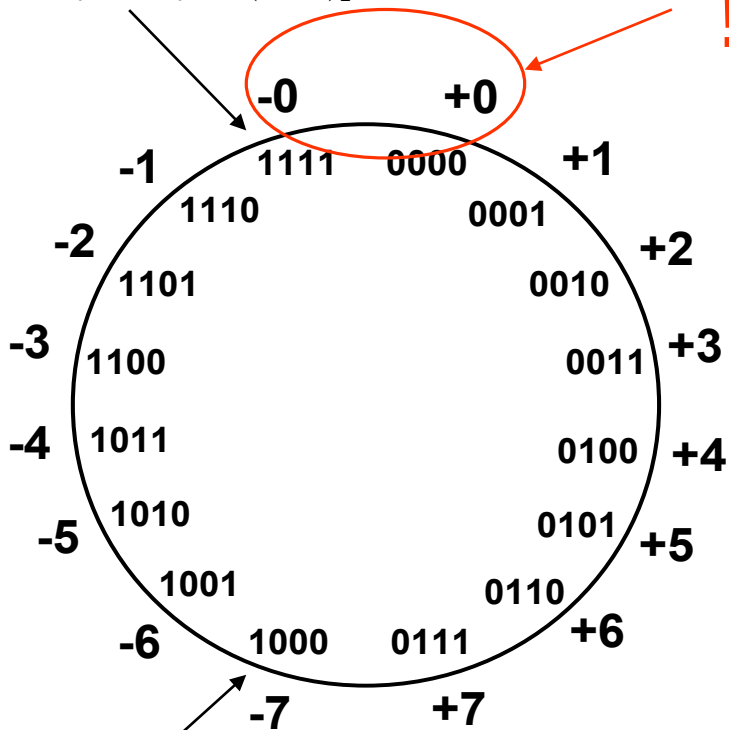
$$a - b = a + \bar{b} - K = a + (2^N - 1 - b) - (2^N - 1) = a - b$$

Mit **$K = K_1 = 2^N - 1$** wird das Einer-Komplement bezeichnet.

2.2.2 Subtraktion

Wertebereich 4 Bit Einer-Komplement ($K_1 = 2^4 - 1 = 15$)

$$\bar{b} = (15 - 0)_{10} = 15_{10} \rightarrow (1111)_2$$



$$\bar{b} = (15 - 7)_{10} = 8_{10} \rightarrow (1000)_2$$

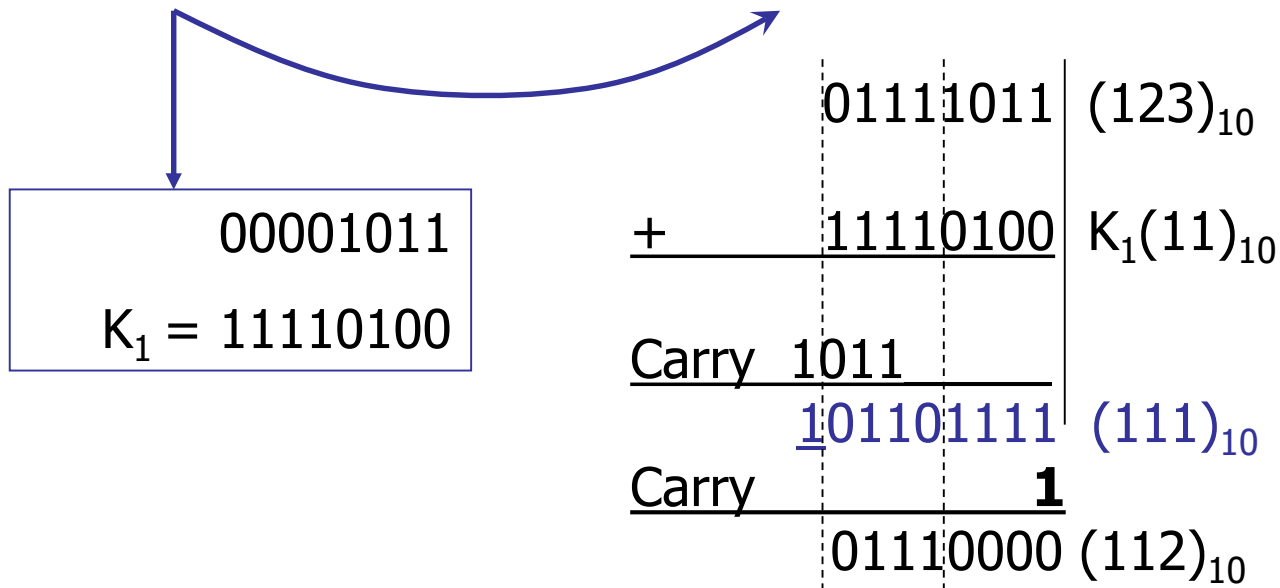
Es fällt auf, dass das Einer-Komplement gerade durch die Negation ($0 \rightarrow 1$ und $1 \rightarrow 0$) der einzelnen Stellen in der Binärzahl entsteht. Nachteilig ist, dass es eine „positive“ und eine „negative“ Null in K₁ entsteht. Deshalb wird das Einer-Komplement in der Praxis nur als Zwischenschritt verwendet.

$$\begin{array}{l} + \\ 0 \ 100 = +4 \\ - \\ 1 \ 011 = -4 \end{array}$$

2.2.2 Subtraktion

Beispiel Einer-Komplement Subtraktion:

$(123)_{10} - (11)_{10} = (112)_{10}$; mit 11110100 als 8 Bit Einer-Komplement



2.2.2 Subtraktion

Das Einer-Komplement besitzt den Nachteil der doppelten „Nullstelle“ und des Carry-Rücklaufs. Beide Eigenschaften führen zu einem entsprechenden Mehraufwand in einer Hardware-Realisierung.

Durch die Verwendung des Zweier-Komplements werden beide Probleme gelöst.

$$\bar{b} = K - b \rightarrow -b = \bar{b} - K, K = 2^N ; \quad N = \text{Stellenanzahl}$$

$$a - b = a + (-b) \stackrel{!}{=} a + \bar{b} - K$$

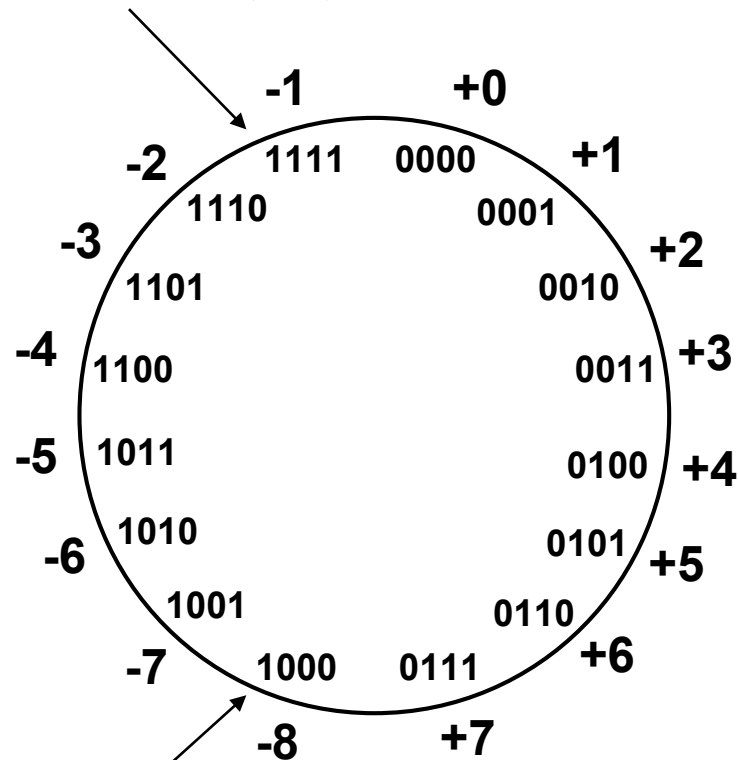
$$a - b = a + \bar{b} - K = a + (2^N - b) - (2^N) = a - b$$

Mit $K = K_2 = 2^N$ wird das Zweier-Komplement bezeichnet ($K_2 = K_1 + 1$).

2.2.2 Subtraktion

Wertebereich 4 Bit Zweier-Komplement ($K_2=2^4=16$)

$$\bar{b} = (16-1)_{10} = 15_{10} \rightarrow (1111)_2$$

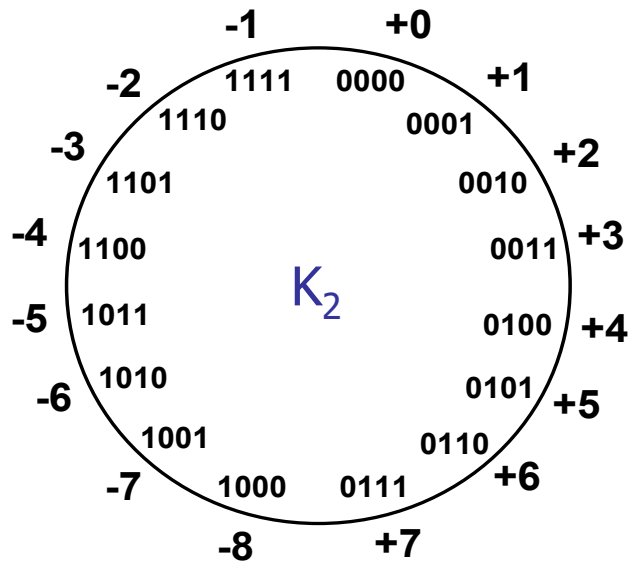


$$\bar{b} = (16-8)_{10} = 8_{10} \rightarrow (1000)_2$$

$$\begin{array}{l} + \\ \swarrow \\ 0 \ 100 = +4 \\ \nwarrow \\ 1 \ 100 = -4 \\ - \end{array}$$

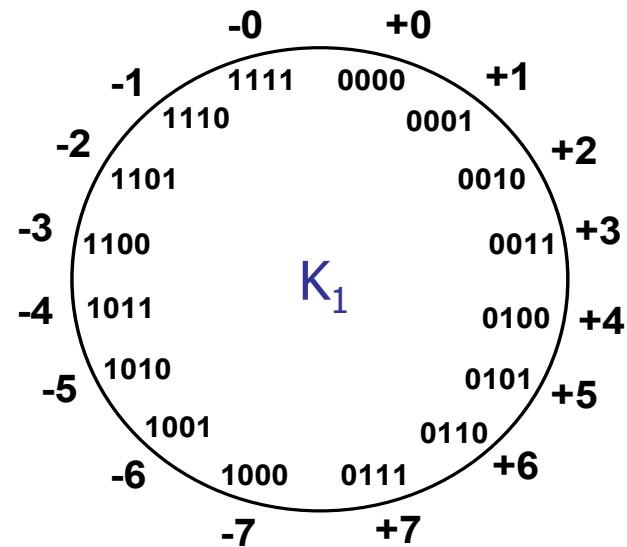
2.2.2 Subtraktion

Vergleich K_2 und K_1



Bereich: $-8 \leq z \leq +7$

$\nearrow +$
 $0\ 100 = +4$
 $\nwarrow -$
 $1\ 100 = -4$
 $-$



Bereich: $-7 \leq z \leq +7$

$\nearrow +$
 $0\ 100 = +4$
 $\nwarrow -$
 $1\ 011 = -4$
 $-$

2.2.2 Subtraktion

Das Zweier-Komplement K_2 lässt sich leicht aus dem Einer-Komplement entwickeln, indem man zum Einer-Komplement eine „Eins“ addiert.

$$\begin{array}{r}
 - (11)_{10} = (11110101)_2; \text{ mit } 11110100 \text{ als 8 Bit „Einer-Komplement“} \\
 \quad \quad \quad + \quad \quad \quad \underline{1} \\
 \quad \quad \quad \underline{11110101} \text{ „Zweier-Komplement“}
 \end{array}$$

Praktische Bestimmung des Zweier-Komplements: Von rechts beginnend werden alle Bits einschließlich der ersten 1 übernommen. Alle nachfolgenden Bits werden negiert.

Beispiel:

0	0	1	1	0	1	1	0
↓	↓	↓	↓	↓	↓	↓	↓
1	1	0	0	1	0	1	0

↓ Übernehmen

↓ Invertieren

2.2.2 Subtraktion

Beispiel Zweier-Komplement Subtraktion:

$(15)_{10} - (11)_{10} = (4)_{10}$; mit 11110101 als 8 Bit Zweier-Komplement

$$\begin{array}{r}
 00001111 \ (15)_{10} \\
 + \quad \underline{11110101 \ K_2(11)_{10}} \\
 \hline
 \text{Carry } \underline{11111111} \\
 \quad \underline{100000100} \\
 \quad 00000100 \ (4)_{10}
 \end{array}$$

Das Abscheiden des MSB (Bit 9) entspricht der Subtraktion von $K_2 = 2^N$.

2.2.3 Multiplikation

Beispiel Multiplikation:

Multiplikand $(11)_{10}$, Multiplikator $(5)_{10}$

$(11)_{10} \cdot (5)_{10}$ als 4 Bit Dualzahlen $\rightarrow$ Produkt ist 2N-stellig

$$\begin{array}{r}
 1011 \cdot 0101 \\
 \hline
 1011 \\
 0000 \\
 1011 \\
 + \quad 0000 \\
 \hline
 \text{Carry} \quad 1 \\
 \hline
 00110111
 \end{array}$$

2.2.3 Multiplikation

Die Multiplikation ist für vorzeichenbehaftete Zahlen (Vz-Zahlen) nach dem gleichen Prinzip möglich (Achtung: Vz symbolisiert das Vorzeichen, es ist nicht das Vorzeichen!).

Hierzu betrachtet man die Vorzeichen des Multiplikanden und des Multiplikators.

Bei **negativen** Vorzeichen eines Operanden, wird dessen Zweier-Komplement multipliziert.

Das Produkt ist positiv, wenn $Vz_1 = Vz_2$, sonst negativ.

2.2.3 Multiplikation

Beispiel:

Vorzeichen-behaftete Multiplikation (Sign-Magnitude):

Multiplikand $(-3)_{10}$, Multiplikator $(5)_{10}$ als 4 Bit Zweier-Komplement

$$(3)_{10} = (0011)_2 \rightarrow (-3)_{10} = (1101)_2 ; (5)_{10} = (0101)_2$$

$$(-3)_{10} = (1101)_2$$

Symbolisierung!

$Vz_1 = 1 \rightarrow \text{negativ}; Vz_2 = 0 \rightarrow \text{positiv}; V_1 \neq V_2 \rightarrow \text{negatives Produkt}$